EthioMorph: Technical Implementation of a Rule-Based Ge'ez Morphological Engine

Esubalew Chekol
College of Technology and Built Environment
Addis Ababa University
NLP Course Project

***Abstract*—Ge'ez verb conjugation follows strict patterns. Given a root like ቀተለ, every form (perfective, imperfective, jussive, imperative) can be computed by rearranging vowels and attaching affixes—no statistical model, but a typed root lexicon (`lexicon.json`) resolves verb class and a handful of edge cases that pure vowel heuristics cannot.**

This paper documents EthioMorph, a Python system that does exactly that. It handles all eight verb types that traditional Ge'ez grammar recognizes: ቀተለ, ቀደሰ, ባረከ, ጦመረ, ሴሰየ, ክህለ, ማሕረከ, and ተንበለ. The core trick: Ethiopic Unicode assigns consecutive codepoints to vowel forms, so changing a vowel is just arithmetic. The rest is template matching, lexicon lookup, and special rules for laryngeal consonants and inherent root-initial ተ/አ sequences.

***Index Terms*—Ge'ez, morphological analysis, NLP, Unicode processing, Semitic morphology, rule-based systems**

I. Introduction

EthioMorph is a Ge'ez conjugator that works from first principles. Not a lookup table with thousands of pre-stored *forms*, and not a neural network trained on text. Conjugation is rule-driven via JSON templates; a compact lexicon maps each attested root to its verb class (`type_a`, `type_d`, `type_tanbala`, etc.).

Why is this even possible? Because Ge'ez is regular. The same root (**ቀተለ**, meaning "kill") always conjugates the same way. There are eight verb types and maybe a dozen special cases for guttural consonants. Once you encode those rules, you can generate or analyze any verb.

A. What the System Does

Ge'ez verbs have two parts: a consonant root (usually 3 letters) and a vowel pattern. The root carries meaning, the pattern carries grammar.

Take **q-t-l** (**ቀተለ**, "kill"). Same three consonants, different vowels:

$$\text{q-t-l} + [\text{1-1-1}] \rightarrow \textbf{ቀተለ} \text{ (past: "he killed")}$$
$$\text{q-t-l} + [\text{yi-1-6-6}] \rightarrow \textbf{ይቅትል} \text{ (present: "he kills")}$$

EthioMorph does this in both directions:

Analysis: **ይቀትሉ** → root: **ቀተለ**, tense: imperfective, subject: 3rd plural masculine

Generation: (**ቀተለ**, imperfective, 3PM) → **ይቀትሉ**

II. Unicode Mathematics for Ethiopic

A. The Ethiopic Block Structure

The Ethiopic Unicode block (U+1200–U+137F) organizes characters in a systematic pattern. Each consonant has 7 forms representing vowel variations:

TABLE I
Unicode Pattern for Consonant ቀ (qäf)

Order	Unicode	Character	Vowel
1	U+1240	ቀ	ä (schwa)
2	U+1241	ቁ	u
3	U+1242	ቂ	i
4	U+1243	ቃ	a
5	U+1244	ቄ	e
6	U+1245	ቅ	(none)
7	U+1246	ቆ	o

B. Vowel Arithmetic

Since vowel forms are consecutive in Unicode, you can do math on them:

$$\text{order}(c) = ((c - \text{base\_offset}) \mod 8) + 1 \tag{1}$$

$$\text{base}(c) = c - (\text{order}(c) - 1) \tag{2}$$

To change a vowel:

$$\text{revowelize}(\text{base}, \text{order}) = \text{base} + (\text{order} - 1) \tag{3}$$

Implementation:

```
def get_vowel_order(char):
    """Extract vowel order (1-7) from
        character."""
    return ORDER_MAP.get(char, 1)

def devowelize(char):
    """Return base consonant (1st order
        form)."""
    return DEVOWELIZATION_MAP.get(char,
        char)

def get_char_by_order(base, order):
    """Apply vowel order to base consonant.
        """
    return REVOWELIZATION_MAP.get((base,
        order), base)
```

Example transformation:

$$\text{devowelize}(\textbf{ቃ}) = \textbf{ቀ}$$
$$\text{get\_vowel\_order}(\textbf{ቃ}) = 4$$
$$\text{get\_char\_by\_order}(\textbf{ቀ}, 6) = \textbf{ቅ}$$

C. Ambiguous Vowel Glyphs

Some Ethiopic characters appear in more than one consonant row. The glyph **ጱ** is shared by both the **ጰ**-row and **ፀ**-row. Flat lookup maps can store only one base per character; EthioMorph pins **ጱ** to the **ጰ**-row after registration (`AMBIGUOUS_VOWEL_DEFAULTS`), matching the normalizer's **ፀ** → **ጰ** homophone map. Both valid bases are recorded in `AMBIGUOUS_VOWELS`.

III. The C1-C2-C3 Radical System

A. Naming the Radicals

C1, C2, C3 refer to the consonants:

- **C1**: First radical (consonant 1)
- **C2**: Second radical
- **C3**: Third radical
- **C4**: Fourth radical (quadriliterals only)

For **ቀተለ**: C1=**ቀ**, C2=**ተ**, C3=**ለ**

B. How Templates Work

Each conjugation form is just a vowel pattern plus optional prefix/suffix:

```python
# Perfective 3rd singular masculine for Type A
template = {
    "vowel_map": {"C1": 1, "C2": 1, "C3": 1},
    "prefix": "",
    "suffix": ""
}
# Result: C1(1st) + C2(1st) + C3(1st) = qä-tä-lä
```

```python
# Imperfective 3rd singular masculine
template = {
    "vowel_map": {"C1": 1, "C2": 6, "C3": 6},
    "prefix": "yi",
    "suffix": ""
}
# Result: yi + C1(1st) + C2(6th) + C3(6th) = yi-qä-t-l
```

C. Putting It Together

IV. Suffix Fusion Rules

A. When Suffixes Collide

When you attach a vowel suffix to a consonant-only ending (6th order), you can't just concatenate them:

$$\text{C}_{\text{6th}} + \text{V}_{\text{suffix}} \rightarrow \text{C}_{\text{order}(V)} \tag{4}$$

Example: **ቅትል** + **ኡ** ("they") ≠ **ቅትልኡ**
Instead: **ቅትል** + **ኡ** → **ቅትሉ**

Algorithm 1 Generate a conjugated word

Require: root (string), tense, subject, verb_type (optional)
Ensure: conjugated word (string)

verb_type ← LEXICON[root] or detect_verb_home(root) or "type_a"
(surface_root, prefix_adj) ← handle_prefixes(root, verb_type) {Skip strip for `type_tanbala` and inherent **ተ**/**አ** roots}
template ← TEMPLATES[verb_type][tense][subject]
vowel_map ← template["vowel_map"]
radicals ← extract_radicals(surface_root)
result ← template["prefix"] + prefix_adj
for each position in ["C1", "C2", "C3", "C4"] **do**
  base ← devowelize(radicals[position])
  order ← vowel_map[position]
  if verb_type == "type_d" and tense == "perfective" and position == "C1" **then**
    if get_vowel_order(surface_root[C1]) == 6 **then**
      order ← 6 {Preserve bare C1, e.g. **ክ** in **ክህለ**}
    end if
  end if
  order ← apply_laryngeal_rules(base, order, position, tense, features)
  result ← result + get_char_by_order(base, order)
end for
result ← result + template["suffix"]
result ← apply_suffix_fusion(result)
return result

B. Fusion Algorithm

```python
def apply_suffix_fusion(stem, suffix):
    """Fuse 6th-order final + vowel suffix."""
    if not suffix:
        return stem

    last_char = stem[-1]
    last_order = get_vowel_order(last_char)

    # Check if stem ends in 6th order (consonant)
    if last_order != 6:
        return stem + suffix

    # Map suffix to target order
    suffix_vowel_map = {
        '\u12A1': 2,  # u-vowel suffix
        '\u12A0': 4,  # a-vowel suffix
        '\u12A5': 5,  # e-vowel suffix
    }

    first_suffix_char = suffix[0]
    if first_suffix_char in suffix_vowel_map:
        target_order = suffix_vowel_map[first_suffix_char]
        base = devowelize(last_char)
        fused = get_char_by_order(base, target_order)
```

```
        return stem[:-1] + fused + suffix
            [1:]

    return stem + suffix
```

V. Verb Home Detection Algorithm

A. The Eight Canonical Verb Types

In Classical Ge'ez grammar, verbs are classified into "homes" (**ቤት**) named after their canonical representative. EthioMorph implements all eight standard verb types:

TABLE II
The Eight Canonical Ge'ez Verb Types (Homes)

#	Type Head	Name	Description
1	**ቀተለ**	Type A	Strong triradical (C1 = 1st order)
2	**ቀደሰ**	Type B	Geminate (C2 doubles in conjugation)
3	**ባረከ**	Type C	Long vowel (C1 = 4th order **ራብዕ**)
4	**ጦመረ**	Type C-O	O-initial (C1 = 7th order **ሳብዕ**)
5	**ሴሰየ**	Weak-Final	Final radical is **የ** (weak)
6	**ክህለ**	Type D	Laryngeal/weak-pattern verbs (`type_d`)
7	**ማሕረከ**	Quadriliteral	Four consonant radicals (`type_mahräka`)
8	**ተንበለ**	Tänbälä	Quadriliteral; initial **ተ** is lexical, not derivational

B. How to Classify a Verb

Each verb type uses different conjugation templates. Classification combines algorithmic vowel-pattern detection with a lexicon override:

- How many radicals? (3 = triradical, 4 = quadriliteral)
- What vowel does C1 have? (5th order = Type B, 4th = Type C, 7th = Type C-O)
- For quadriliterals: is C2 at 6th order? (Tänbälä pattern, e.g. **ተንበለ**)
- Any laryngeals (**ሀ, ሐ, ኀ, አ, ዐ**) or weak consonants (**ወ, የ**)? (stored as *features*, not always the class)
- Lexicon entry when present (e.g. **ክህለ** → `type_d`, **ፈጸመ** → `type_d`)

TABLE III
Algorithmic Classification Rules (before lexicon override)

Condition	Type	Example
|radicals| = 4, C2 order = 6	Tänbälä	**ተንበለ**
|radicals| = 4, else	Quadriliteral	**ማሕረከ**, **ተጋደለ**
C1 order = 7	Type C-O	**ጦመረ**
C1 order = 5	Type B	**ቀደሰ**
C1 order = 4	Type C	**ባረከ**
C3 ∈ {weak}	Weak-Final	**ሴሰየ**
Default (3 radicals)	Type A	**ቀተለ**, **አመረ**
Lexicon match	As listed	**ክህለ** → `type_d`

C. Detection Algorithm

The conjugator resolves the final class as `LEXICON[root]` or `detect_verb_home(root).type` or ``type_a''. Roots such as **ክህለ** and **ፈጸመ** are listed as `type_d` in the lexicon even when vowel heuristics alone would yield Type A.

Algorithm 2 Figure out which verb type this is (`detect_verb_home`)

Require: root (string with vowels)
Ensure: (type, features)

```
skeleton ← consonant_skeleton(root)
radicals ← len(skeleton)
c1_order ← get_vowel_order(root[0])
c2_order ← get_vowel_order(root[1]) {if present}
features ← detect_laryngeal_and_hollow(skeleton)
if radicals == 4 then
    if c2_order == 6 then
        return ("type_tanbala", features) {ተንበለ}
    else
        return ("type_mahräka", features) {ማሕረከ, ተጋደለ}
    end if
end if
if c1_order == 5 then
    return ("type_b", features) {ቀደሰ}
else if c1_order == 4 then
    return ("type_c", features) {ባረከ}
else if c1_order == 7 then
    return ("type_c_o", features) {ጦመረ}
end if
return ("type_a", features) {default; ክህለ uses type_d via lexicon}
```

D. Feature Detection

Some roots have special properties that affect conjugation:

```
LARYNGEALS = {'h', 'H', 'x', 'a', 'A'}  # Gutturals
WEAK_CONSONANTS = {'w', 'y'}

def detect_features(root, radicals):
    features = {}

    # Check for laryngeal consonants
    for rad in radicals:
        base = devowelize(rad)
        if base in LARYNGEAL_BASES:
            features['has_laryngeal'] =
                True
            break

    # Check for hollow verb (weak C2)
    if len(radicals) >= 2:
        c2_base = devowelize(radicals[1])
        if c2_base in WEAK_BASES:
            features['is_hollow'] = True

    # Check for weak initial
    c1_base = devowelize(radicals[0])
    if c1_base in WEAK_BASES:
        features['weak_initial'] = True

    return features
```

VI. Laryngeal Vowel Shift Rules

A. Gutturals Are Picky

Laryngeal consonants (**ሀ, ሐ, ኀ, አ, ዐ**) don't accept certain vowels. When you try to apply a 1st order vowel to a laryngeal, it shifts:

$$\text{Laryngeal} + \text{1st order} \rightarrow \text{Laryngeal} + \text{4th order} \quad (5)$$

B. Implementation

```
def apply_laryngeal_rules(char, position,
    features):
    """Shift␣vowels␣near␣laryngeal␣
        consonants."""
    if not features.get('has_laryngeal'):
        return char

    base = devowelize(char)
    order = get_vowel_order(char)

    # Laryngeals prefer 4th order in
        jussive
    if is_laryngeal(base) and order == 1:
        return get_char_by_order(base, 4)

    return char
```

VII. Conjugator: Generation Edge Cases

A. Inherent vs. Derivational Prefixes

During *analysis*, the stemmer strips tense markers (**ይ**, **ተ**, **አ**, etc.). During *generation*, the conjugator must *not* strip leading **ተ**/**አ**/**ነ** when they belong to the lexical root itself. Stripping is allowed only for derived stem templates (`type_a_passive`, `*_causative`, etc.).

```
# PREFIX_LOOKALIKES = (asn, aste, te, a, ne
    ) in Ge'ez script

def should_strip_prefix(root, verb_type):
    if verb_type == "type_tanbala":
        return False  # tenbele: te- is C1,
            not passive
    if verb_type and not
        is_derived_stem_type(verb_type):
        if root.startswith(
            PREFIX_LOOKALIKES):
            return False  # e.g. amare,
                tagedele
    return True
```

Examples: (**ተንበለ**, perfective, 3sm) → **ተንበለ** (identity); (**ተንበለ**, jussive, 3sm) → **ይተንብል** (not the erroneous double-marked **ይተነብል** produced when **ተ-** was stripped).

B. Type D Perfective Templates

Type D verbs (**ክህለ** "be able", **ፈጸመ** "finish") use a distinct perfective vowel map: C1 defaults to 1st order, C2 to 6th (bare consonant), C3 to 1st.

```
"type_d": {
  "perfective": {
    "3sm": {"vowel_map": {"C1": 1, "C2": 6,
        "C3": 1}},
    "3sf": {"suffix": "t", "vowel_map": {"
        C1": 1, "C2": 6, "C3": 1}}
  }
}
```

Guttural roots often store C1 at 6th order in the lexicon (**ክ** in **ክህለ**). The conjugator preserves that input order when C1 is already bare, so (**ክህለ**, perfective) → **ክህለ** while (**ፈጸመ**, perfective, 3sf) → **ፈጽመት** (C2 **ጸ** → **ፅ**).

C. Hollow Verb Surface Variants

Hollow verbs (`hollow_w`, `hollow_y`) admit *collapsed* imperative surfaces where the middle weak radical disappears and C1 carries a distinctive vowel order. For **ሐወረ** "to go," the full imperative is **ሕወር**, but collapsed forms **ሖር** and the ultra-short **ሖ** also occur (C1 at 7th order signals the hidden **ወ**).

The conjugator emits these as non-destructive `variants` alongside the primary generated word:

```
{
  "word": "<ho-wer>",            # primary
      imperative (full surface)
  "variants": ["<hor>", "<ho>"],  #
      collapsed alternates
  "derivation": { ... }
}
```

The simple expand API mirrors this as `result["variants"][tense][person]`. The UI (Morpher, Matrix, Compare, Root Tree) renders primary and collapsed forms together so users see both the template output and attested short surfaces.

Perfective hollow verbs collapse differently: (**ሐወረ**, perfective, 3sm) → **ሐረ** with C2 dropped and C1 at 1st order. The stemmer reconstructs such two-radical skeletons back to the lexicon root by inserting the middle weak consonant when the expanded form is listed as `hollow_w` or `hollow_y`.

VIII. Stemmer: Going Backwards

The Stemmer takes a conjugated word and figures out its root. It's the reverse of generation. Prefix stripping here applies to *tense and stem markers on inflected forms*, not to the inherent root-initial radicals guarded during generation.

A. The Steps

1) Normalize spelling (Ge'ez has some redundant characters)
2) Strip prefixes (**ይ**, **ተ**, **አ**, etc.)
3) Strip suffixes (**ኡ**, **ከ**, etc.)
4) Get the consonant skeleton
5) Restore any weak consonants that disappeared
6) Resolve to lexicon canonical orthography
7) Figure out tense/person from what you stripped
8) Classify the verb type

B. Canonical Root Resolution

Homophone normalization maps variant spellings to a single row for matching (**ሐ** → **ሀ**, **ሕ** → **ሀ**, etc.). That aids reconstruction, but the *reported* root should use the lexicon citation form when one exists. For example, **ሖ**, **ሖር**, and **ሕወር** all analyze to **ሐወረ**—not **ሀወረ**—because the lexicon lists the **ሐ**-row root and surface imperatives use **ሖ** (7th order on **ሐ**), not **ሆ** (7th order on **ሀ**).

```
def canonicalize_root(root):
    if root in LEXICON:
        return root
    norm = normalize_geez(root)
    if norm in LEXICON_NORMALIZED:
        return LEXICON_NORMALIZED[norm]
    if norm in HOLLOW_W_LOOKUP:
        return HOLLOW_W_LOOKUP[norm]
    if norm in HOLLOW_Y_LOOKUP:
        return HOLLOW_Y_LOOKUP[norm]
    return root
```

Two-radical perfective stems such as **ሐረ** and **ቀመ** are expanded by inserting **ወ** or **የ** when the resulting triliteral appears in the typed hollow-verb lists. Vowel-order signals (3rd, 5th, or 7th order on C1) take precedence over bare lexicon matches for ambiguous two-letter forms such as **ጌሰ** (collapsed **ገየሰ**) versus the unrelated root **ገሰ**.

C. Checking Prefixes

You have to check longer prefixes first, or you'll make wrong matches:

```
PREFIXES = [
    # Stem IV (Causative-Passive) - check
        first
    'yaste', 'taste', 'naste', 'laste',
    # Stem III (Causative)
    'yas', 'tas', 'nas', 'las',
    'ya', 'ta', 'na', 'a',
    # Stem II (Passive)
    'yit', 'tit', 'te',
    # Stem I (Basic)
    'yi', 'ti', 'ni', 'li'
]
# Sorted by length descending
PREFIXES.sort(key=len, reverse=True)
```

D. Pattern Identification

```
def identify_verb_pattern(stem,
    stripped_prefixes):
    """Determine␣grammatical␣pattern␣from␣
        affixes."""

    # Check stem markers in priority order
    if 'aste' in stripped_prefixes:
        return {'stem': 4, 'name': '
            Causative-Passive'}

    if 'as' in stripped_prefixes or
        has_causative_vowel(stem):
        return {'stem': 3, 'name': '
            Causative'}

    if 'te' in stripped_prefixes or 'yit'
        in stripped_prefixes:
        return {'stem': 2, 'name': 'Passive
            '}

    return {'stem': 1, 'name': 'Basic'}
```

IX. Data Structures

A. Lexicon JSON

Each root entry supplies the verb class used by the conjugator (overriding `detect_verb_home` when present):

```
{
  "roots": [
    {"root": "<keHle>", "type": "type_d", "
        meaning": "to␣be␣able"},
    {"root": "<feTSeMe>", "type": "type_d",
         "meaning": "to␣finish"},
    {"root": "<tenbele>", "type": "
        type_tanbala", "meaning": "to␣
        intercede"}
  ]
}
```

In the repository, `root` values are Ge'ez strings (**ክህለ**, **ፈጸመ**, **ተንበለ**, etc.).

B. Templates JSON

```
{
  "type_a": {
    "perfective": {
      "3sm": {
        "vowel_map": {"C1": 1, "C2": 1, "C3
            ": 1},
        "prefix": "",
        "suffix": ""
      },
      "3sf": {
        "vowel_map": {"C1": 1, "C2": 1, "C3
            ": 4},
        "prefix": "",
        "suffix": "t"
      }
      // ... 10 person forms
    },
    "imperfective": {
      "3sm": {
        "vowel_map": {"C1": 1, "C2": 6, "C3
            ": 6},
        "prefix": "yi",
        "suffix": ""
      }
      // ...
    }
    // jussive, imperative, derived forms
  },
  "type_d": {
    "perfective": {
      "3sm": {"vowel_map": {"C1": 1, "C2":
          6, "C3": 1}},
      "3sf": {"vowel_map": {"C1": 1, "C2":
          6, "C3": 1}, "suffix": "t"}
    }
  }
}
```

C. Lookup Maps

Three precomputed dictionaries make character lookups instant (with one documented ambiguity for ጰ):

```
# Character -> Base consonant
DEVOWELIZATION_MAP = {
    '\u1240': '\u1240',  # qe -> qe (already base)
    '\u1241': '\u1240',  # qu -> qe
    '\u1242': '\u1240',  # qi -> qe
    # ... all 300+ characters
}

# Character -> Vowel order (1-7)
ORDER_MAP = {
    '\u1240': 1,  # qe = 1st order
    '\u1241': 2,  # qu = 2nd order
    # ...
}

# (Base, Order) -> Character
REVOWELIZATION_MAP = {
    ('\u1240', 1): '\u1240',
    ('\u1240', 2): '\u1241',
    ('\u1240', 3): '\u1242',
    # ...
}
```

The glyph ጰ (U+13BA) is registered in both the ጸ-row and ፀ-row; `AMBIGUOUS_VOWELS` records both bases, with ጸ pinned as the default in `AMBIGUOUS_VOWEL_DEFAULTS`.

X. System Architecture

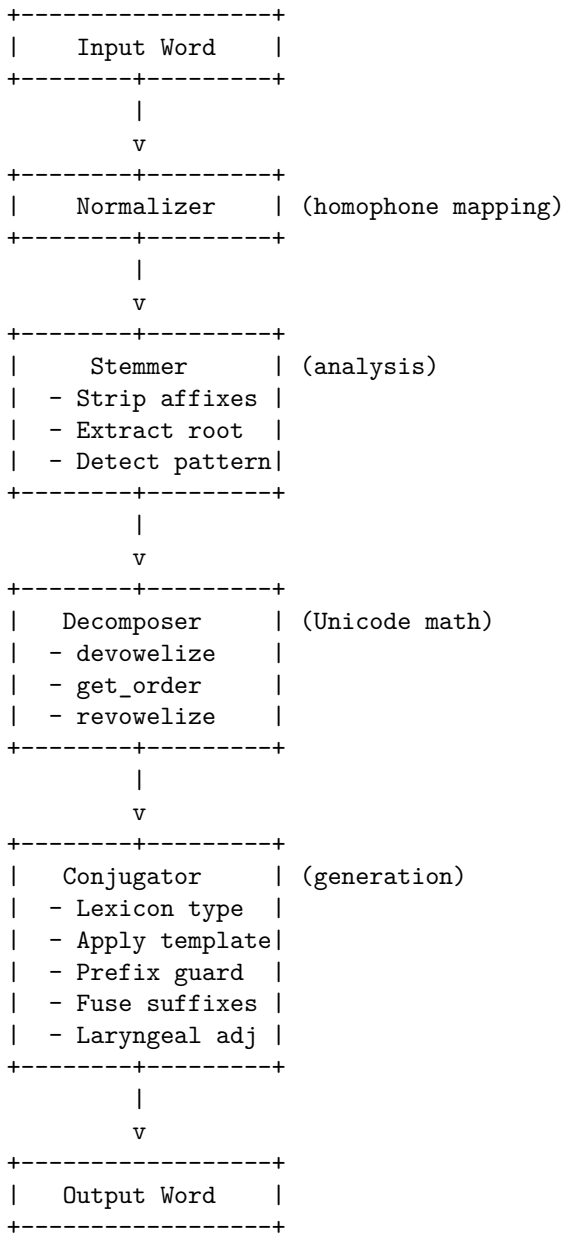

Fig. 1. How a word flows through the system

XI. API Usage

A. Analysis Endpoint

```
POST /api/xray
{
  "word": "yiqatlu"
}

Response:
{
  "root": "qatala",
  "pattern": "imperfective",
  "stem": 1,
  "person": "3pm",
  "verb_type": "type_a"
}
```

B. Generation Endpoint

```
POST /api/morph
{
  "root": "qatala"
}

Response:
{
  "perfective": {
    "3sm": "qatala",
    "3sf": "qatalat",
    ...
  },
  "imperfective": {...},
  "jussive": {...}
}
```

XII. Observations

Ge'ez morphology turns out to be more regular than expected. The Unicode trick (vowel forms are consecutive codepoints) turns what seems like a complex linguistic operation into basic arithmetic. Most of the code applies JSON templates, consults a typed lexicon, and handles edge cases: guttural vowel shifts, Type D perfective C2 muting, Tänbälä roots whose initial ተ must not be treated as a passive prefix, and ambiguous glyphs such as ጰ.

The system isn't complete. It handles the eight main verb types and basic stems, but derived forms and obscure patterns remain partial. Still, it conjugates most roots attested in the lexicon with rule-level accuracy on the regression suites for prefix look-alikes and Type D perfective forms.

Code: `github.com/Esubaalew/EthioMorph`